ConectaStore

Sistema de Recomendação de Produtos Baseado em Grafos, implementado em Rust

---

Relatório Técnico · Parte Teórica

Estudo de caso: MegaStore (comércio eletrônico)

Autor(a): Nome do(a) aluno(a)

Repositório: https://github.com/Kayke-ns/megastore

Outubro de 2026

Pergunta central

Como utilizar grafos e estruturas de dados complementares para gerar recomendações relevantes em um catálogo com grande volume de produtos?

Resposta curta: lista de adjacência ponderada + BFS com propagação de relevância, apoiada por HashMap, HashSet e BinaryHeap. 1 milhão de produtos em ~1,6 ms por consulta.

Sumário

1. Introdução . . . 3
2. Análise do problema . . . 3
   - Requisitos derivados . . . 3
3. Modelagem do sistema como grafo . . . 4
   - Por que não direcionado? . . . 4
   - Por que heterogêneo e ponderado? . . . 4
4. Vértices, arestas, pesos e tipo de grafo . . . 5
   - Vértices . . . 5
   - Arestas e pesos . . . 5
   - Classificação do grafo . . . 5
5. Lista de adjacência versus matriz de adjacência . . . 6
   - Detalhe de implementação . . . 7
6. Estruturas de dados escolhidas e justificativas . . . 7
7. Algoritmo de recomendação . . . 8
   - 7.1 Por que BFS? . . . 8
   - 7.2 Propagação de relevância . . . 8
   - 7.3 Prevenção de recomendações duplicadas . . . 9
   - 7.4 Recomendação a partir de produto . . . 9
   - 7.5 Cold start . . . 9
   - 7.6 Explicação (DFS com poda) . . . 9
   - Resultado na base de demonstração . . . 9
8. Análise de complexidade de tempo e espaço . . . 10
   - Memória por elemento . . . 10
9. Desempenho e escalabilidade . . . 11
   - Metodologia . . . 11
   - Leitura dos resultados . . . 11
   - Efeito do corte de fan-out . . . 12
   - Estratégias de escalabilidade . . . 12
10. Implementação, testes e evidências de execução . . . 13
   - Testes . . . 13
   - Capturas de tela . . . 13
11. Conclusão . . . 19

Referências . . . 20

1. Introdução

Este relatório descreve o projeto **ConectaStore**, um sistema de recomendação de produtos para a MegaStore desenvolvido em Rust. O sistema representa clientes, produtos e categorias como vértices de um grafo e as compras, avaliações, interesses e similaridades como arestas ponderadas. A partir desse grafo, um algoritmo de busca em largura com propagação de relevância encontra produtos ligados ao comportamento de cada cliente.

O documento cobre a análise do problema, a modelagem, a comparação entre lista e matriz de adjacência, a justificativa de cada estrutura de dados, o algoritmo, a análise de complexidade e os resultados de desempenho medidos na implementação (Parte Prática). Todas as decisões descritas aqui estão no código do repositório e são verificadas por 36 testes automatizados.

2. Análise do problema

A MegaStore comercializa milhões de itens em categorias muito diferentes (eletrônicos, vestuário, alimentos, livros, móveis, decoração). O sistema atual recomenda apenas os mais vendidos ou itens da mesma categoria. Essa abordagem tem três falhas:

- **É genérica:** todos os clientes recebem praticamente a mesma lista; a recomendação não depende do histórico.
- **Ignora relações entre categorias:** quem compra um notebook costuma precisar de mouse e monitor (mesma categoria), mas também de cadeira ergonômica (outra categoria). Filtrar só pela categoria esconde essa venda cruzada.
- **Não usa os dados que a empresa já tem:** históricos de compra, avaliações e interesses existem, mas não estão estruturados para representar *conexões* entre consumidores.

O efeito no negócio é a perda de venda cruzada, de aumento do ticket médio e de fidelização. Do ponto de vista técnico, o problema é de **relacionamento**: a pergunta "o que este cliente provavelmente quer?" é respondida olhando quem se parece com ele e o que essas pessoas compraram. Relações de muitos para muitos, com intensidades diferentes, são exatamente o que um grafo ponderado representa bem.

Requisitos derivados

Requisito	Consequência no projeto
Catálogo com milhões de produtos	Estruturas com memória proporcional aos dados (O(V+E)) e consultas por ID em O(1).
Recomendação em tempo de navegação	Custo por consulta limitado e quase independente do tamanho do catálogo.
Recomendações relevantes e variadas	Combinar compras, avaliações, interesses e similaridade, com pesos.
Sem itens repetidos ou já comprados	Exclusão explícita e estrutura de pontuação com chave única por produto.
Clientes novos (cold start)	Usar interesses declarados e completar com mais vendidos.
Explicabilidade	Mostrar o caminho do grafo que justifica cada recomendação.

3. Modelagem do sistema como grafo

O sistema é modelado como um **grafo heterogêneo** (vários tipos de vértice e de aresta), **não direcionado** e **ponderado**. A figura abaixo mostra o esquema: três tipos de vértice e cinco tipos de relação.

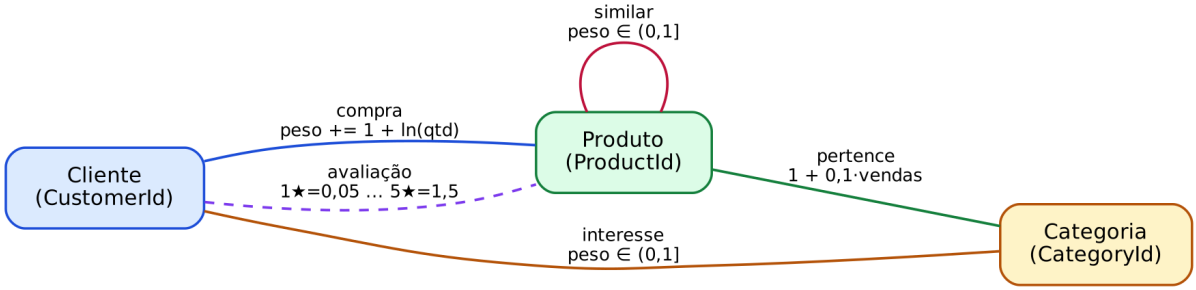


Figura 1. Esquema do grafo: tipos de vértice e de aresta com a regra de peso de cada relação.

A Figura 2 mostra um recorte do grafo real da demonstração. A cliente Ana comprou um notebook, um mouse e um livro de Rust. Bruno e Diego compraram o mesmo notebook e também o teclado e a cadeira ergonômica; Carla comprou o livro de Rust e o livro de algoritmos. Os produtos destacados em amarelo são alcançados a partir de Ana em 2 ou 3 saltos e se tornam candidatos à recomendação.

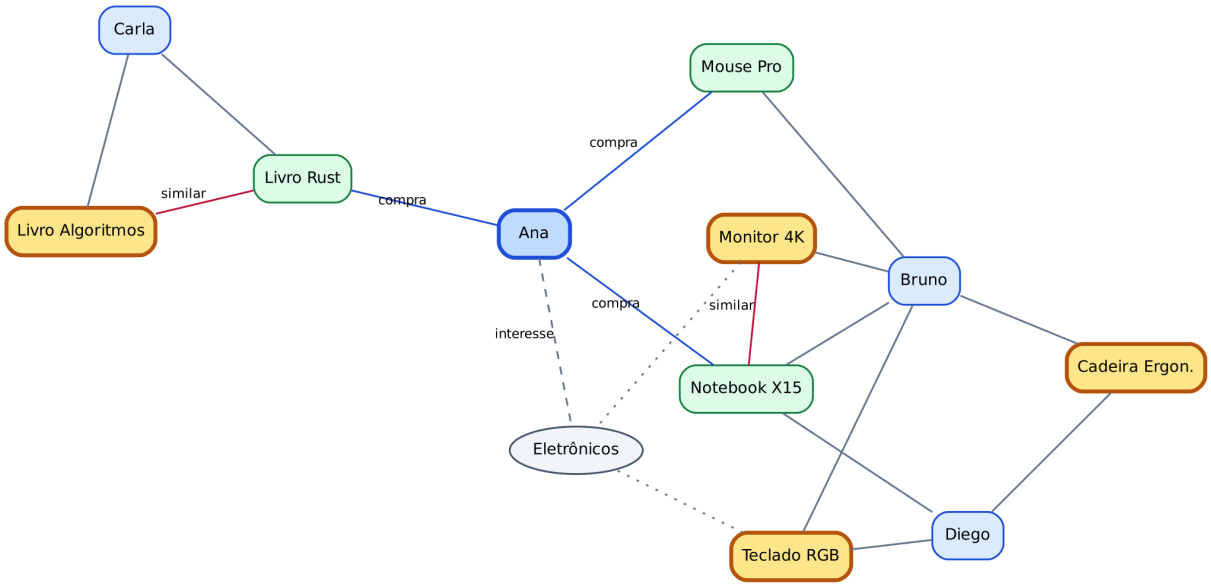


Figura 2. Recorte do grafo de demonstração. Em amarelo, produtos que o algoritmo recomenda para Ana.

Por que não direcionado?

As relações são simétricas para fins de recomendação: "Ana comprou o notebook" e "o notebook foi comprado por Ana" são o mesmo fato, e o algoritmo precisa percorrê-lo nos dois sentidos (cliente para produto e produto para clientes). Cada aresta é armazenada nas duas pontas, com um ponteiro para a gêmea, o que mantém o percurso em qualquer direção em O(grau).

Por que heterogêneo e ponderado?

Uma compra repetida diz mais do que uma compra única; uma avaliação de 5 estrelas diz mais do que uma de 2; pertencer à mesma categoria é um sinal mais fraco do que ter sido comprado pelas mesmas pessoas. Pesos por aresta e fatores por tipo de aresta permitem expressar essas diferenças sem criar grafos separados.

4. Vértices, arestas, pesos e tipo de grafo

Vértices

Tipo	Identificador	Representa	Volume (cenário XL)
Cliente	CustomerId (u32)	Comprador da loja	500.000
Produto	ProductId (u32)	Item do catálogo	1.000.000
Categoria	CategoryId (u32)	Agrupamento de produtos	1.000

Internamente, cada vértice recebe um índice denso `NodeIdx` (0, 1, 2, ...). Um `HashMap<NodeKey, NodeIdx>` traduz o identificador de negócio para esse índice uma única vez; a partir daí o percurso usa apenas acesso direto a vetores, que é rápido e amigável ao cache.

Arestas e pesos

Aresta	Liga	Peso	Fator no algoritmo	Atualização
Compra	Cliente, Produto	1 + ln(qtd) por evento	1,00	acumula a cada compra
Avaliação	Cliente, Produto	1★ 0,05 · 2★ 0,2 · 3★ 0,6 · 4★ 1,0 · 5★ 1,5	1,00	substitui (vale a última)
Similar	Produto, Produto	(0, 1], curadoria ou conteúdo	0,90	substitui
Interesse	Cliente, Categoria	(0, 1], declarado pelo cliente	0,70	substitui
Categoria	Produto, Categoria	1 + 0,1 · (1 + ln qtd) por venda	0,35	acumula com as vendas

- O logaritmo na compra evita que um pedido de 100 unidades domine a relação.
- Avaliações ruins quase não propagam relevância: se Ana deu 1 estrela a um produto, pessoas parecidas com ela por causa desse produto contam pouco.
- A aresta produto-categoria ganha peso com as vendas. Isso faz o corte de fan-out (seção 7) de uma categoria preferir os produtos mais vendidos dela.
- Não existem arestas paralelas do mesmo tipo: registrar a mesma compra duas vezes reforça a aresta existente.

Classificação do grafo

Propriedade	Escolha	Motivo
Direção	Não direcionado	Relações simétricas, percorridas nos dois sentidos.
Ponderação	Ponderado (f32)	Intensidade diferente para cada relação.
Tipos	Heterogêneo (3 vértices, 5 arestas)	Integra todos os sinais em uma estrutura só.

Propriedade	Escolha	Motivo
Laços	Proibidos	Produto similar a si mesmo não tem significado (erro SelfLoop).
Arestas paralelas	Uma por par e tipo	Evita duplicidade; o peso acumula.
Densidade	Esparso ($E/V^2 \approx 3{,}4 \cdot 10^{-6}$ no XL)	Determina a escolha da lista de adjacência.

5. Lista de adjacência versus matriz de adjacência

Há duas representações clássicas para um grafo. Na **matriz de adjacência**, uma tabela V × V guarda o peso de cada par de vértices (zero quando não há aresta). Na **lista de adjacência**, cada vértice guarda apenas a lista dos vizinhos que de fato possui.

Operação	Lista de adjacência	Matriz de adjacência
Memória	**O(V + E)**	$O(V^2)$
Verificar se (u, v) existe	O(min(grau u, grau v))	**O(1)**
Percorrer vizinhos de u	**O(grau u)**	O(V)
Inserir aresta	O(min grau) com verificação de duplicata	**O(1)**
Remover aresta	O(min grau) + O(1) com swap_remove	**O(1)**
Inserir vértice	**O(1) amortizado**	$O(V^2)$ (realocar a matriz)
Vários tipos de aresta	**campo kind na própria aresta**	uma matriz por tipo (5×)
BFS completa	**O(V + E)**	$O(V^2)$

O grafo da MegaStore é extremamente esparso: um cliente compra dezenas de produtos, não milhões. Como o algoritmo de recomendação passa quase todo o tempo *percorrendo vizinhos*, a lista vence na operação que importa (O(grau) contra O(V)) e também na memória. A tabela a seguir compara a memória medida da lista com a memória que uma matriz densa de f32 exigiria ($V^2 \times 4$ bytes), sem contar as 5 matrizes que os tipos de aresta pediriam.

Cenário	Vértices	Arestas	Lista (medida)	Matriz ($V^2 \cdot 4$ B)	Razão
XS	1.520	7.196	389 KB	8,8 MB	23×
S	15.050	75.212	4,1 MB	864,0 MB	212×
M	150.200	756.944	39,7 MB	84,0 GB	2.168×
L	750.500	3.802.134	195,6 MB	2,0 TB	10.987×
XL	1.501.000	7.603.874	390,8 MB	8,2 TB	21.992×

Com 100 mil produtos a matriz já exigiria 84 GB, mais do que a memória de um servidor comum; com 1 milhão, 8 TB. A lista ocupa 391 MB. Vale notar que, no grafo minúsculo da demonstração (34 vértices), a matriz ocuparia 4,5 KB e a lista 5,9 KB: a matriz só compensa em grafos pequenos ou densos, ou quando a consulta dominante é "existe a aresta (u, v)?". Nenhum desses casos descreve a MegaStore. **Decisão: lista de adjacência.**

Detalhe de implementação

```
pub struct Edge {          // 16 bytes por meia aresta
    pub to: NodeIdx,       // vizinho (u32)
    pub rev: u32,          // posição da aresta gêmea em adj[to]
    pub weight: f32,
    pub kind: EdgeKind,    // compra, avaliação, interesse, categoria, similar
}
pub struct Graph {
    keys: Vec<NodeKey>,                    // índice -> identidade de negócio
    index: HashMap<NodeKey, NodeIdx>,      // identidade -> índice, O(1)
    adj: Vec<Vec<Edge>>,                   // lista de adjacência
    weight_sum: Vec<f32>,                  // soma dos pesos por vértice
    edge_count: usize,
}
```

O campo rev é o que torna a lista completa para atualização: ao encontrar a aresta (varrendo a lista do vértice de **menor** grau), sabemos exatamente onde está a gêmea, então mudar o peso ou remover a aresta nas duas pontas não exige uma segunda busca nem um índice auxiliar. Na remoção, swap_remove move o último elemento para o buraco em O(1) e corrige o rev da aresta movida. Um teste faz 400 inserções e 300 remoções aleatórias e verifica esse invariante após cada operação.

6. Estruturas de dados escolhidas e justificativas

Estrutura (Rust)	Uso	Justificativa	Alternativa descartada
HashMap<ProductId, Product>	Catálogo	Cadastro e consulta por ID em O(1) médio. Medido: 18 a 138 ns.	Vec ordenado + busca binária: O(log n) e inserção O(n).
HashMap<CategoryId, Vec<ProductId>>	Índice por categoria	Lista uma categoria sem varrer o catálogo.	Filtrar o catálogo inteiro: O(n).
Vec<Vec<Edge>>	Lista de adjacência	O(V+E) de memória, vizinhos contíguos (cache), O(grau) por vértice.	Matriz: O(V^2) (seção 5).
HashMap<NodeKey, NodeIdx>	Tradução de IDs	Desacopla ID de negócio do índice do vetor; O(1).	Usar o ID como índice: IDs esparsos desperdiçariam memória.
HashSet<NodeIdx>	Visitados e excluídos	Teste de pertinência O(1); impede reexpandir vértices e recomendar itens comprados.	Vec<bool> de tamanho V: O(V) por consulta só para zerar.
HashMap<NodeIdx, f64>	Pontuação	Uma chave por produto: acumula caminhos e impede duplicatas.	Vec de candidatos: exigiria deduplicação O(n log n).
BinaryHeap (min-heap de tamanho k)	Top-k e mais vendidos	O(n log k) e O(k) de memória.	Ordenar todos: O(n log n).
VecDeque	Fila da BFS de menor caminho	push/pop nas pontas em O(1).	Vec com remove(0): O(n).
Vec como pilha	DFS iterativa	Sem recursão profunda, sem estouro de pilha.	DFS recursiva em grafo grande.
select_nth_unstable	Corte de fan-out	Seleciona as F arestas mais fortes em O(grau).	Ordenar a lista: O(grau log grau).

Também foram escolhidos tipos de tamanho fixo e compactos: identificadores em u32 (metade de um u64, suficientes para 4 bilhões de itens), pesos em `f32` nas arestas e `f64` apenas nos acumuladores de pontuação, onde a precisão importa. Preços são guardados em centavos (u64) para não sofrer com arredondamento de ponto flutuante. Erros são tipados (`StoreError`) e devolvidos em `Result`, nunca por pânico.

7. Algoritmo de recomendação

O enunciado pede BFS, DFS ou outro algoritmo justificado. O ConectaStore usa os dois: **BFS por níveis com propagação de relevância** para gerar as recomendações e **DFS com poda** para explicá-las.

7.1 Por que BFS?

Relevância diminui com a distância: o produto comprado por quem comprou o mesmo que você (3 saltos) é um sinal forte; algo a 8 saltos é praticamente aleatório. A BFS visita o grafo exatamente em ordem de distância, então basta parar no nível 3 para limitar o trabalho ao que importa. Uma DFS iria fundo em um único ramo antes de olhar os vizinhos próximos, o que é o oposto do desejado aqui.

7.2 Propagação de relevância

A BFS clássica só marca visitados. Aqui cada vértice carrega uma *massa*. O cliente começa com 1,0. A cada nível, a massa de um vértice é distribuída entre seus vizinhos na proporção do peso de cada aresta, normalizada pela soma dos pesos do vértice, e multiplicada pelo fator do tipo de aresta. Todo produto que recebe massa acumula pontuação, com decaimento de 0,85 por nível. A ideia é a mesma de um passeio aleatório truncado (como no PageRank personalizado) e da filtragem colaborativa item a item.

```
função recomendar(origem, k, D = 3, F = 256, decaimento = 0,85):
    excluídos  ← {origem} ∪ produtos comprados ou avaliados pela origem
    visitados  ← {origem}
    fronteira  ← {origem: 1,0}
    pontuação  ← HashMap vazio
    fator_nível ← 1,0
    repita D vezes:
        próxima ← HashMap vazio
        para cada (v, massa) em fronteira:
            para cada aresta e entre as F mais fortes de v:
                m ← massa · e.peso / soma_pesos(v) · fator(e.tipo)
                se e.destino é produto e não está em excluídos:
                    pontuação[e.destino] += m · fator_nível
                se e.destino não está em visitados:
                    próxima[e.destino] += m
        visitados ← visitados ∪ chaves(próxima)
        fronteira ← próxima;  fator_nível ← fator_nível · decaimento
    resultado ← top_k(pontuação, k)             // min-heap de tamanho k
    se |resultado| < k: completar com mais vendidos fora de excluídos
    devolver resultado
```

Elementos importantes:

- **Normalização pelo grau ponderado:** um best-seller com 50 mil compradores divide sua massa entre todos eles, então não domina todas as recomendações (problema clássico de popularidade).

- **Visitados:** cada vértice é *expandido* no máximo uma vez, o que garante término e limita o custo. Mesmo assim, um produto alcançado por vários caminhos acumula a massa de todos eles: mais caminhos, mais relevância.
- **Corte de fan-out (F = 256):** vértices de grau muito alto (categorias, best-sellers) só propagam pelas F arestas mais pesadas, escolhidas com `select_nth_unstable` em O(grau). Isso torna o custo de uma consulta praticamente independente do tamanho do catálogo (seção 9).
- **Determinismo:** a fronteira é ordenada por índice e o desempate do top-k é pelo menor ID, de modo que a mesma entrada sempre gera a mesma saída (testado).

7.3 Prevenção de recomendações duplicadas

A duplicidade é evitada em quatro camadas: (1) o grafo não aceita arestas paralelas do mesmo tipo, então compras repetidas reforçam a aresta existente; (2) a pontuação é um HashMap com uma única entrada por produto; (3) um HashSet de IDs já emitidos protege a montagem da lista final, inclusive quando ela é completada com mais vendidos; (4) o conjunto de excluídos impede recomendar a própria origem e itens que o cliente já comprou ou avaliou. Na demonstração, três compras do mesmo notebook mantêm o total em 65 arestas, e o top-15 de Ana tem 15 itens distintos, nenhum já comprado.

7.4 Recomendação a partir de produto

A mesma função funciona a partir de um vértice de produto ("quem comprou X também comprou"): os caminhos produto → clientes → produtos e produto → similar dão os itens relacionados. Para o Notebook Gamer X15, o primeiro resultado é o Monitor 4K (similar e comprado junto por Bruno), seguido da Cadeira Ergonômica, comprada por dois dos três compradores do notebook: venda cruzada entre categorias que o sistema antigo não encontraria.

7.5 Cold start

Um cliente sem compras, mas com interesse declarado (Hugo, interesse em Móveis), recebe recomendações pelo caminho cliente → categoria → produto, já priorizando os mais vendidos da categoria graças ao peso da aresta de categoria. Se não houver nenhuma conexão, a lista é completada com os mais vendidos (heap de tamanho k), marcados com a origem "mais vendidos".

7.6 Explicação (DFS com poda)

Para mostrar *por que* um item foi recomendado, o sistema procura o caminho de maior contribuição entre o cliente e o produto com uma DFS limitada a 3 saltos (32 arestas mais fortes por vértice). Como a massa só diminui ao longo de um caminho, qualquer ramo cuja massa já seja menor que a do melhor caminho encontrado é descartado (*branch and bound*). A DFS é adequada aqui porque precisamos enumerar caminhos completos, não distâncias. Exemplo real da demonstração:

```
[cliente] Ana
   └─(compra)─▶ [produto] Livro: Rust na Prática
   └─(similar)─▶ [produto] Livro: Algoritmos e Estruturas
força do caminho: 0.0247
```

Resultado na base de demonstração

#	Produto recomendado para Ana	Categoria	Score	Caminho dominante
1	Monitor 27" 4K	Eletrônicos	0,0549	notebook → similar; também Bruno
2	Livro: Algoritmos e Estruturas	Livros	0,0537	livro de Rust → similar; também Carla, Fábio
3	Teclado Mecânico RGB	Eletrônicos	0,0486	notebook/mouse → Bruno e Diego
4	Cadeira Ergonômica	Móveis	0,0194	notebook → Bruno e Diego (venda cruzada)
5	Luminária de Mesa LED	Casa e Decoração	0,0136	livro/café → Fábio

8. Análise de complexidade de tempo e espaço

Notação: V vértices, E arestas, P produtos, d(v) grau do vértice v, k tamanho da lista, D profundidade (3), F fan-out (256), S quantidade de produtos candidatos encontrados em uma consulta, R vértices visitados em uma consulta.

Operação	Tempo	Espaço extra
Cadastrar produto / cliente / categoria	O(1) médio	O(1)
Consultar produto por ID	O(1) médio	O(1)
Listar produtos de uma categoria	O(1) + O(c) para percorrer	O(1)
Buscar por nome	O(P · m), m = tamanho do nome	O(resultados)
Registrar compra / avaliação / interesse	O(min(d(u), d(v)))	O(1) amortizado
Remover conexão	O(min(d(u), d(v)))	O(1)
Recomendar (propagação BFS)	$O(\Sigma_{v \in R} d(v)) \leq O(V + E)$; com corte: $O(\Sigma_{v \in R}$ (d(v) seleção + min(d(v), F)))	O(R + S)
Top-k	O(S log k)	O(k)
Completar com mais vendidos	$O(P_{vendidos}$ log k)	O(k)
Explicar (DFS com poda)	$O(F_e^D)$ no pior caso, $F_e = 32$ (≤ 32.768 caminhos)	O(D)
Menor caminho (BFS)	O(V + E)	O(V)
Alcance limitado (DFS)	O(V + E)	O(V)
Armazenamento total		O(P + V + E)

Sobre a recomendação: sem o corte, o pior caso teórico é percorrer o componente inteiro, O(V + E), porque cada vértice é expandido uma vez e cada aresta examinada uma vez. Na prática a profundidade 3 limita R aos vizinhos dos vizinhos dos vizinhos, e o corte F limita quantas arestas cada um desses vértices propaga. A única parte que ainda depende do grau é a seleção das F mais fortes em vértices muito conectados, feita em tempo linear. Por isso o tempo medido cresce de forma sublinear com o catálogo (seção 9).

Memória por elemento

Cada aresta ocupa 2 × 16 = 32 bytes (uma meia aresta em cada ponta). Cada vértice custa cerca de 50 bytes (chave, entrada no HashMap, cabeçalho do Vec, soma de pesos). No cenário XL: 7,6 milhões de arestas × 32 B ≈ 243 MB, mais 1,5 milhão de vértices × ~50 B ≈ 75 MB, mais folga de capacidade dos vetores: 391 MB medidos. O catálogo (nomes e preços) fica em um HashMap separado, proporcional a P.

9. Desempenho e escalabilidade

Metodologia

O módulo `benchmark.rs` gera lojas sintéticas com um gerador pseudoaleatório determinístico (semente fixa): cada cliente tem uma ou duas categorias favoritas, faz em média 8 compras, avalia 30% delas, e a popularidade dos produtos dentro de cada categoria é assimétrica (poucos itens vendem muito), como em uma loja real. Para cada cenário são medidos: tempo de construção, 200 mil consultas por ID, 500 recomendações para clientes aleatórios (média, p95 e máximo) e 500 recomendações por produto. Ambiente: contêiner Linux, 2 vCPUs, 7 GB de RAM, compilação `--release` com LTO.

Cenário	Produtos	Clientes	Arestas	Montagem	Busca ID	Rec. cliente média	Rec. cliente p95	Rec. produto
XS	1.000	500	7.196	2 ms	18 ns	166 µs	226 µs	126 µs
S	10.000	5.000	75.212	22 ms	19 ns	530 µs	769 µs	429 µs
M	100.000	50.000	756.944	444 ms	63 ns	829 µs	1,19 ms	600 µs
L	500.000	250.000	3.802.134	3,46 s	118 ns	1,34 ms	2,08 ms	1,03 ms
XL	1.000.000	500.000	7.603.874	7,28 s	138 ns	1,56 ms	2,39 ms	1,01 ms

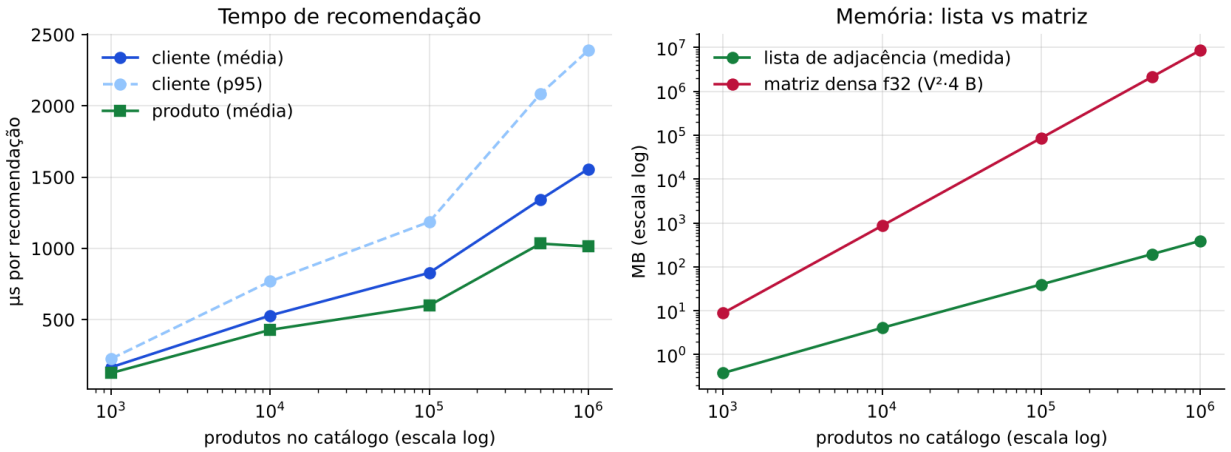


Figura 3. Tempo por recomendação (esquerda) e memória da lista versus matriz (direita), ambos medidos na implementação.

Leitura dos resultados

- O catálogo cresceu **1000×** (de 1 mil para 1 milhão de produtos) e o tempo médio de recomendação cresceu cerca de **9×** (de 0,17 para 1,6 ms). O p95 ficou abaixo de 2,4 ms em todos os cenários.

- A consulta por ID continua na casa dos nanossegundos. O aumento de 18 para 138 ns vem de falhas de cache quando o HashMap deixa de caber na memória cache do processador, e não do algoritmo, que segue O(1).
- A construção é linear: cerca de 7 µs por produto com todos os seus relacionamentos.
- A memória cresce linearmente (391 MB no XL), enquanto a matriz cresceria quadraticamente (8 TB).

Efeito do corte de fan-out

Para justificar o valor F = 256, o benchmark compara o top-5 obtido com vários cortes contra o resultado exato (sem corte) para 200 clientes do cenário M:

max_fanout	16	64	256 (padrão)	1024	ilimitado
Tempo médio	232 µs	441 µs	894 µs	2.832 µs	2.909 µs
Top-5 igual ao exato	91,9%	98,2%	99,2%	100%	100%

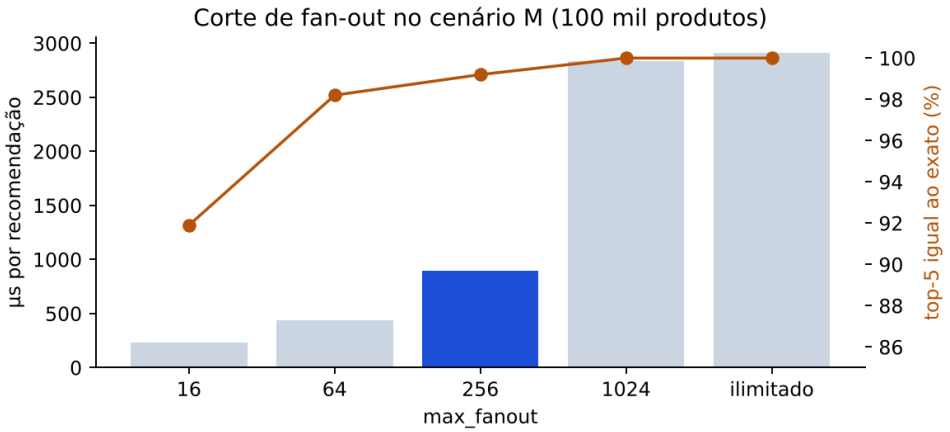


Figura 4. Custo e qualidade em função do corte de fan-out.

Com F = 256 a consulta fica 3,3 vezes mais rápida e 99,2% das recomendações coincidem com o cálculo exato. É um parâmetro de configuração: pode ser reduzido em momentos de pico de acesso.

Estratégias de escalabilidade

- **Leitura concorrente:** a recomendação só lê o grafo (`&self`), então várias consultas podem rodar em paralelo compartilhando o grafo atrás de um `Arc`; escritas podem ser agrupadas em lotes.
- **Pré-cálculo:** recomendações dos clientes mais ativos podem ser calculadas em lote fora do horário de pico e guardadas em cache, invalidadas quando o cliente compra algo novo.
- **Particionamento:** acima de centenas de milhões de arestas, o grafo pode ser dividido por região ou por grupo de categorias, ou migrado para um banco de grafos.
- **Compactação:** como os vizinhos já estão em vetores contíguos, uma versão somente leitura em formato CSR (Compressed Sparse Row) eliminaria a folga de capacidade dos vetores.
- **Busca textual:** a busca por nome é linear e seria a primeira a trocar por um índice invertido.

10. Implementação, testes e evidências de execução

A Parte Prática está organizada em módulos com responsabilidades separadas. A fachada `MegaStore` integra catálogo, cadastros e grafo; o módulo de recomendação não conhece catálogo nem preços, só o grafo.

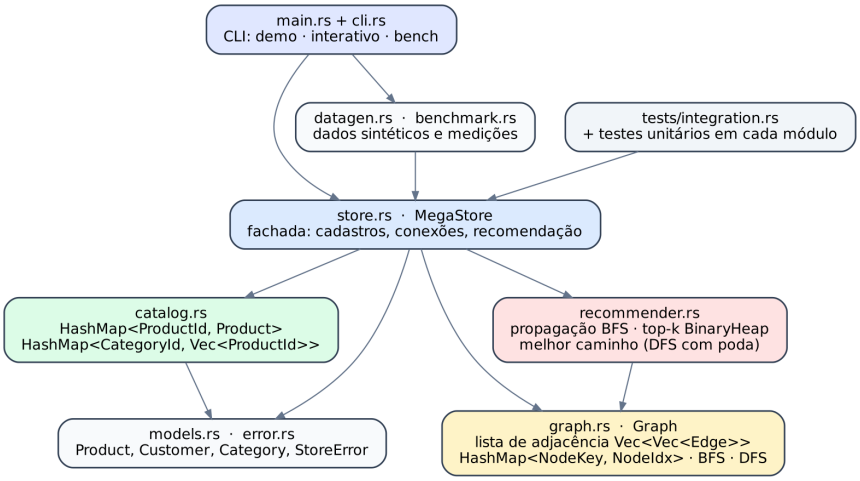


Figura 5. Arquitetura da solução.

Testes

São 22 testes unitários (dentro dos módulos) e 14 de integração (`tests/integration.rs`, usando só a API pública). `cargo clippy` não aponta nenhum aviso.

Grupo	O que é verificado
Catálogo	cadastro, consulta por ID, rejeição de ID duplicado, índice por categoria, busca por nome, atualização de preço
Grafo	vértices idempotentes, upsert sem arestas paralelas, acumulação e substituição de pesos, remoção, invariante dos ponteiros rev após 700 operações aleatórias, BFS de menor caminho, alcance por DFS
Algoritmo	ranking colaborativo, limite de profundidade, ordenação e desempate do top-k, seleção de fan-out, melhor caminho
Integração	fluxo completo, erros de cadastro e referências inválidas, ausência de duplicatas e de itens comprados para todos os clientes, compra repetida, ordem das pontuações, cold start, recomendação por produto, devolução, mudança de interesse, explicação, estatísticas, determinismo, volume de 20 mil produtos

Capturas de tela

As imagens a seguir são capturas da execução real do projeto (saída do terminal), geradas com os comandos indicados no topo de cada uma.

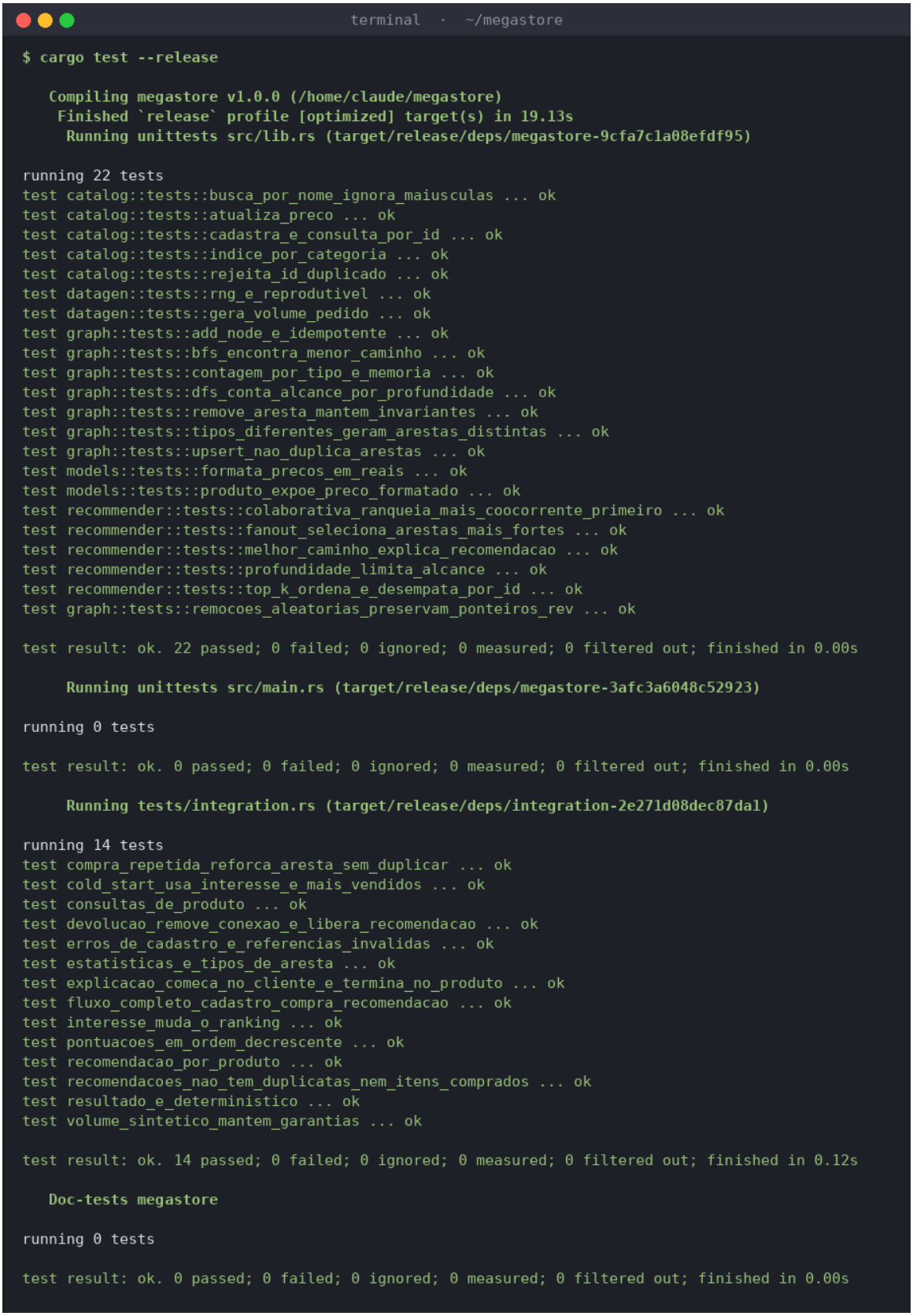


Figura 6. cargo test: 36 testes aprovados.

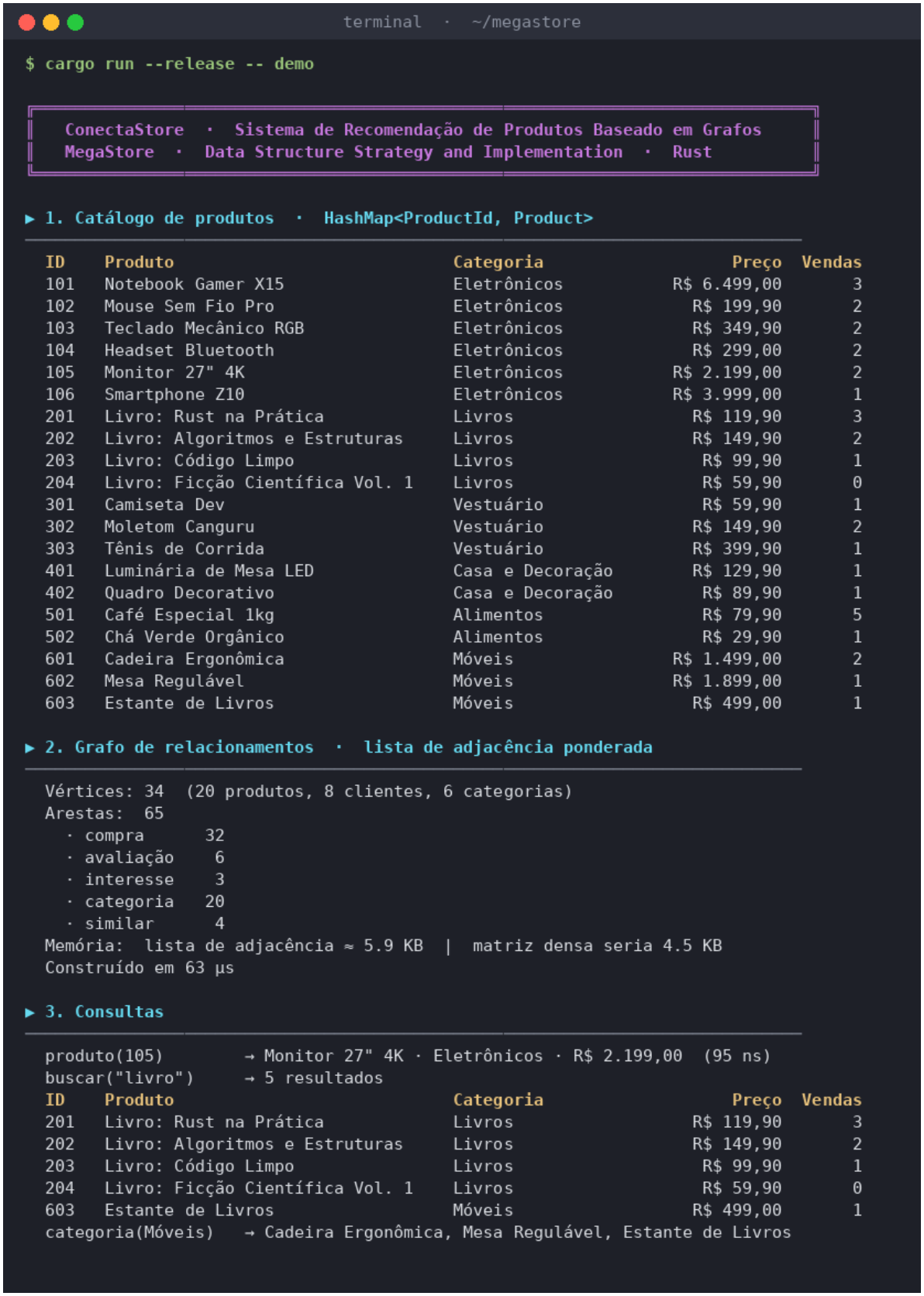


Figura 7. Demonstração: catálogo, estatísticas do grafo e consultas.

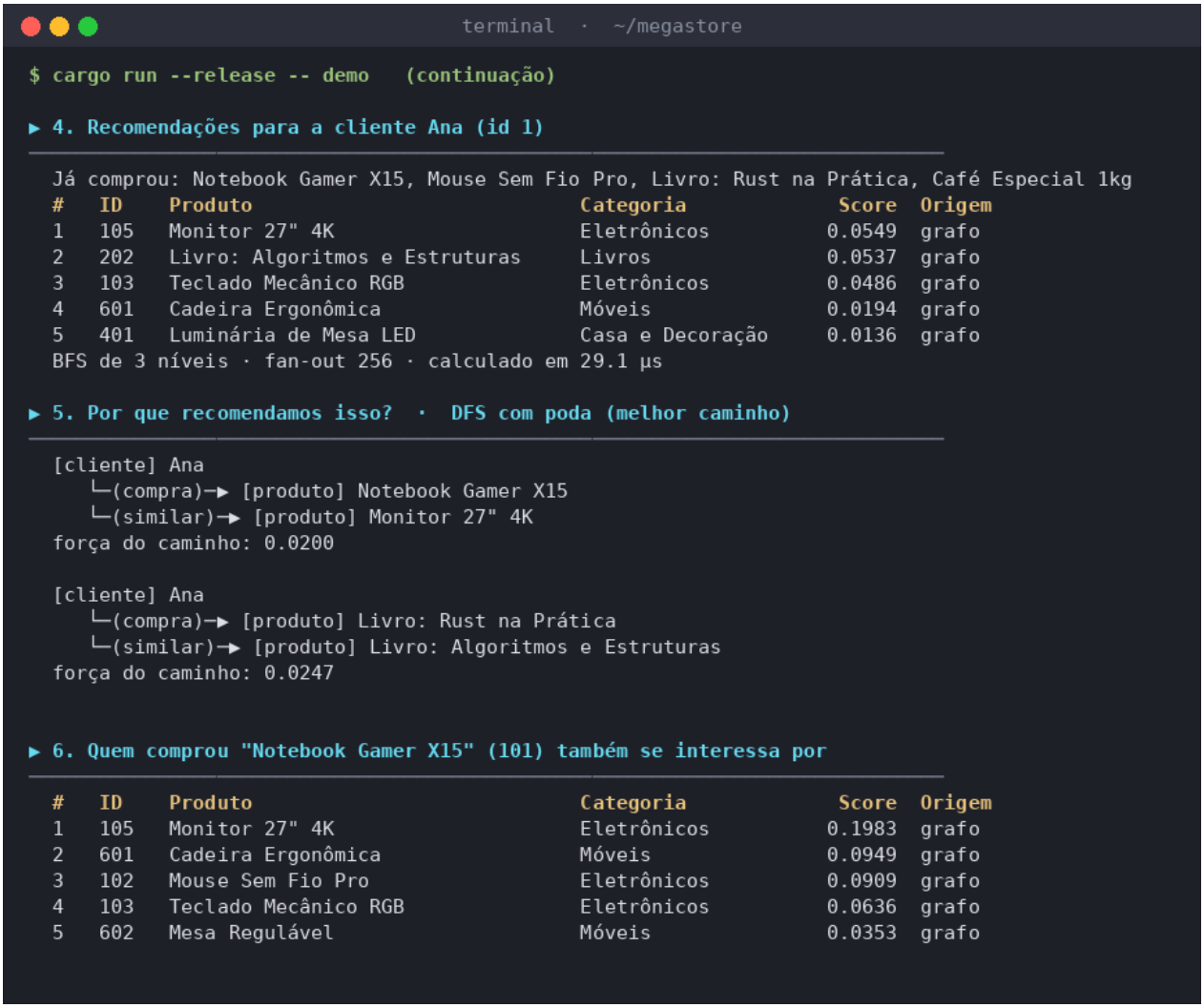


Figura 8. Recomendações para Ana, explicação por caminho e produtos relacionados.

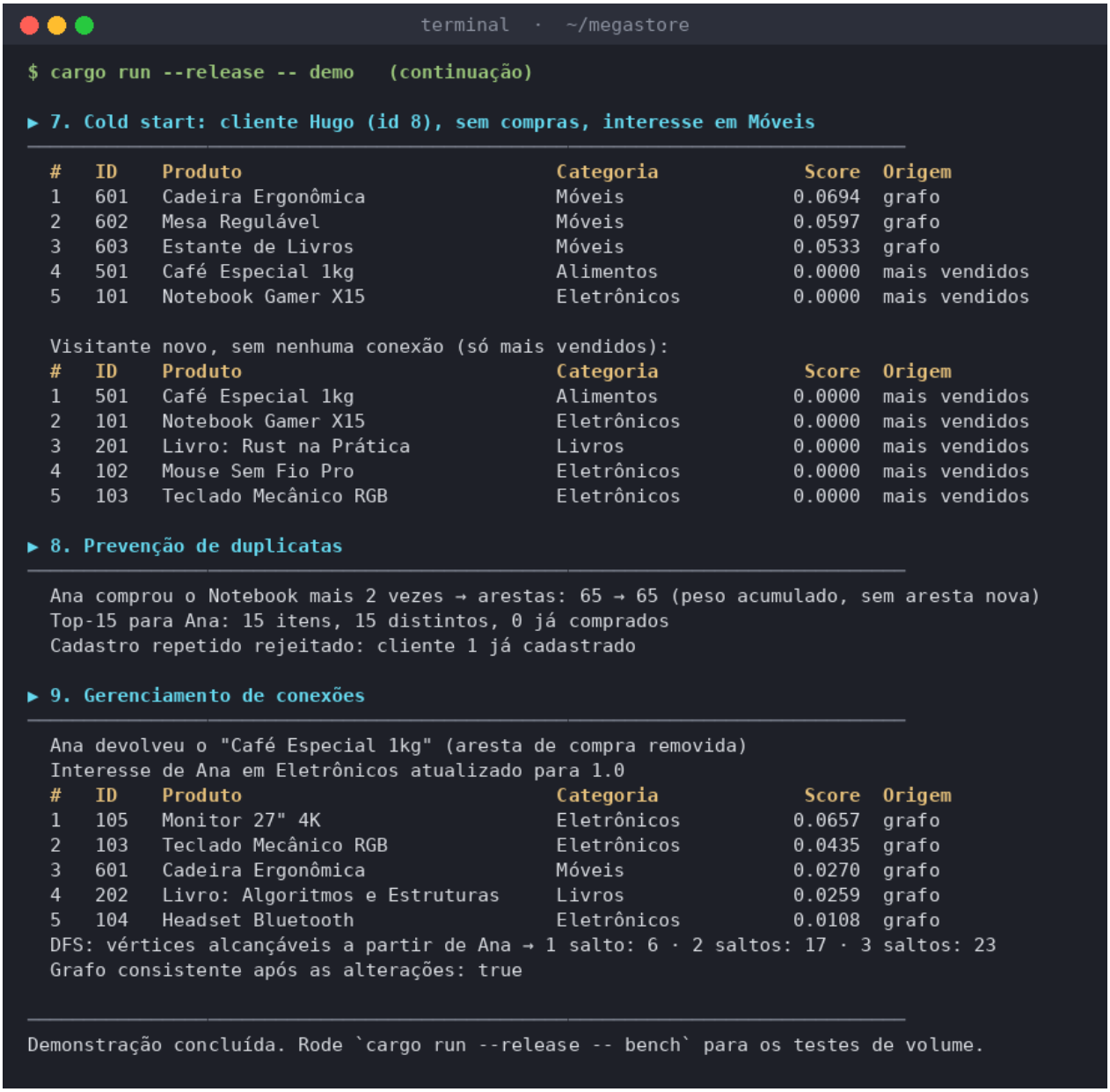


Figura 9. Cold start, prevenção de duplicatas e gerenciamento de conexões.

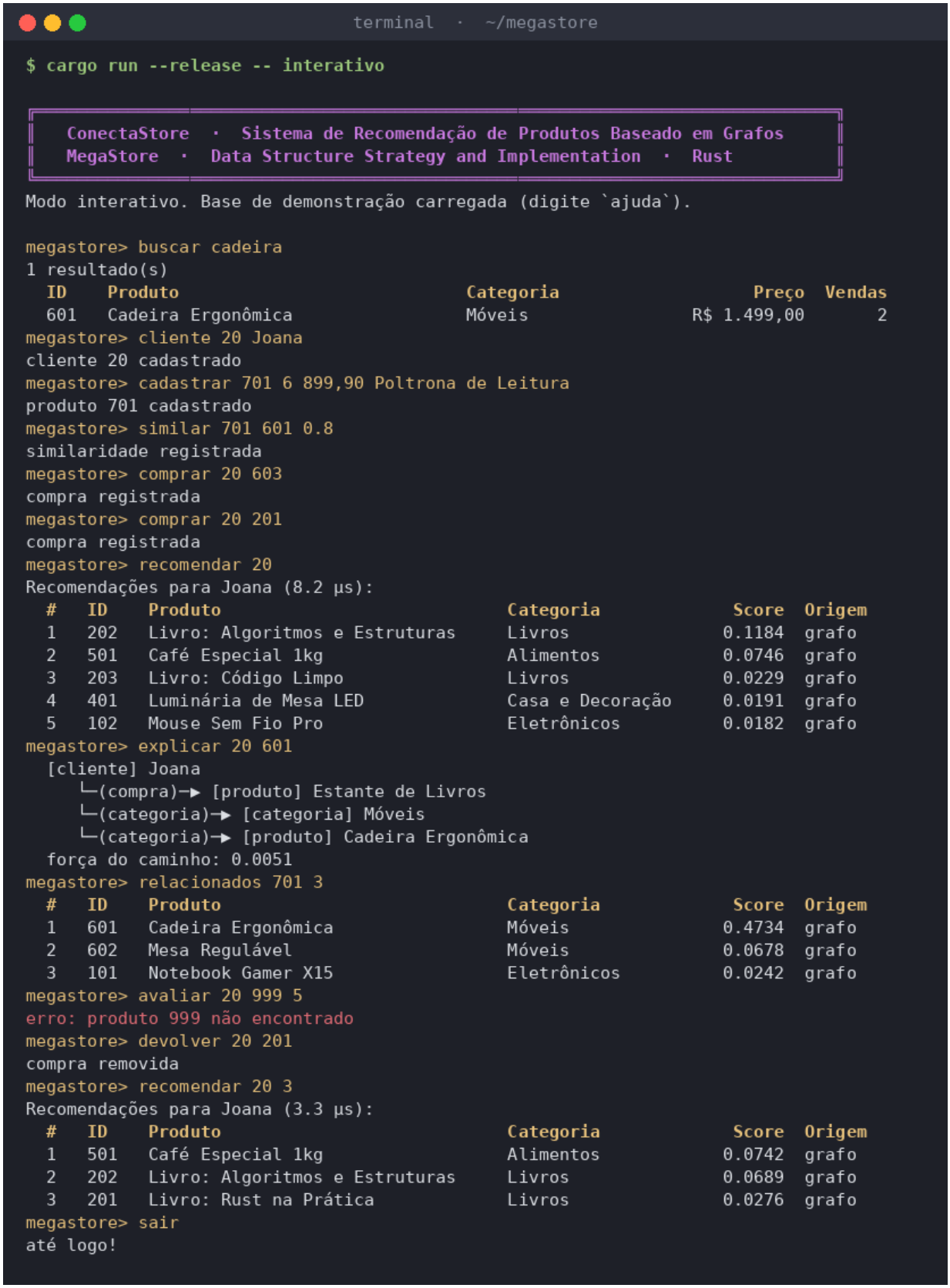


Figura 10. Modo interativo: cadastro, compra, recomendação, explicação e devolução.

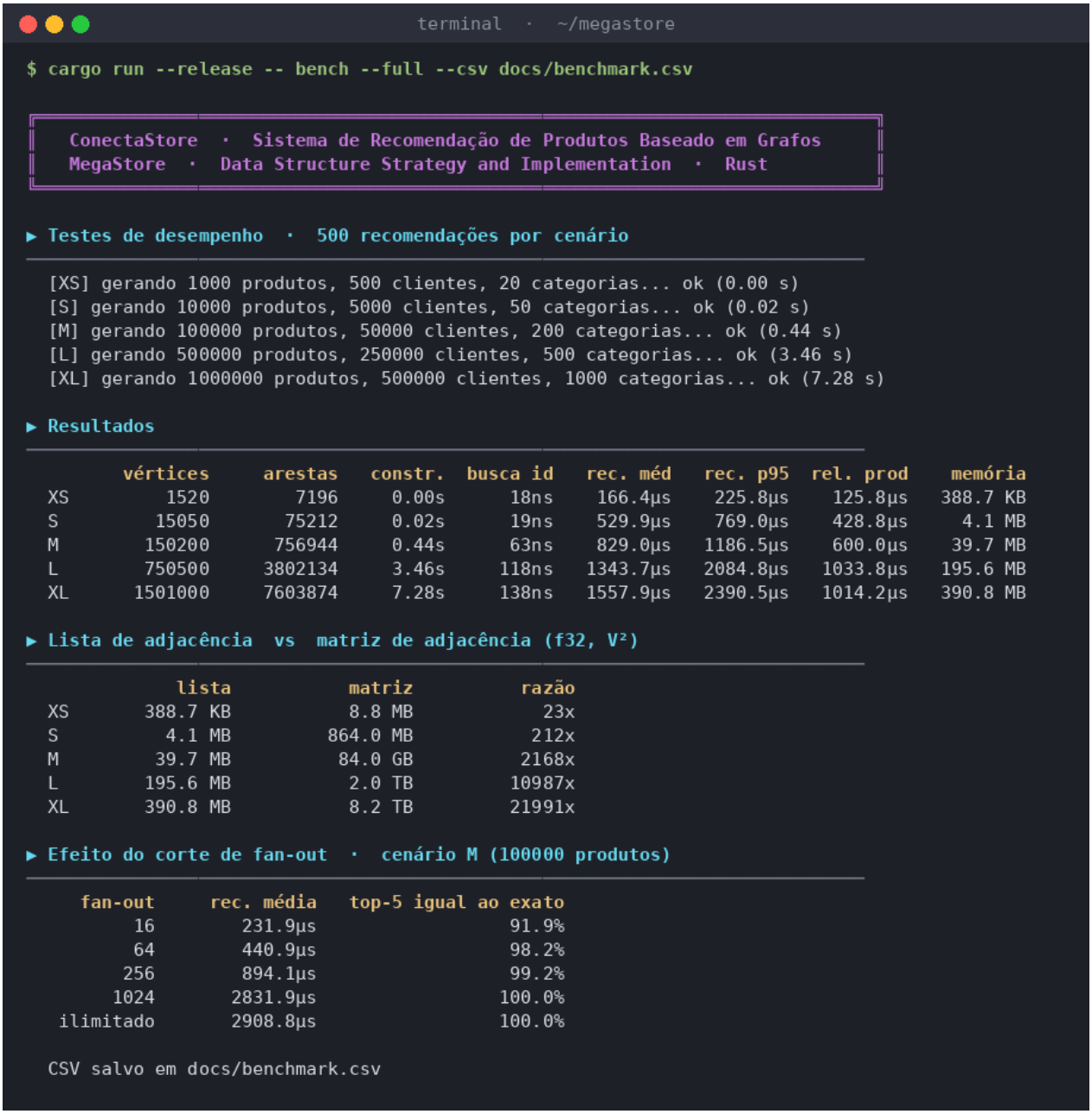


Figura 11. Testes de desempenho com 1 mil a 1 milhão de produtos.

11. Conclusão

Como utilizar grafos e estruturas de dados complementares para gerar recomendações relevantes em um catálogo com grande volume de produtos?

Representando clientes, produtos e categorias como vértices de um grafo esparso, ponderado e heterogêneo, guardado em **lista de adjacência**, e percorrendo-o com uma **BFS limitada** que propaga relevância proporcional aos pesos. O grafo captura o que a abordagem atual da MegaStore ignora: quem se parece com quem e como as categorias se conectam pelo comportamento real dos clientes. Isso gera recomendações personalizadas e vendas cruzadas, como a cadeira ergonômica sugerida a quem comprou um notebook.

As estruturas complementares tornam a solução viável em escala: o **HashMap** dá acesso aos produtos e vértices em tempo constante; o **HashSet** controla visitados e exclusões; o **HashMap de pontuações** e o **BinaryHeap** de tamanho k garantem uma lista ordenada e sem duplicatas em O(S log k); a **fila** e a **pilha** suportam BFS e DFS. Com o corte de fan-out, uma recomendação para um catálogo de 1 milhão de produtos leva cerca de 1,6 ms usando 391 MB, onde uma matriz de adjacência exigiria 8 TB.

Como próximos passos, os pesos e fatores definidos manualmente podem ser calibrados com testes A/B, a busca textual pode ganhar um índice invertido e o grafo pode ser servido em paralelo para muitas consultas simultâneas.

Referências

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Algoritmos: teoria e prática*. 3. ed. Rio de Janeiro: Elsevier, 2012. Capítulos 6 (heaps), 11 (tabelas hash) e 22 (algoritmos elementares em grafos).

LINDEN, G.; SMITH, B.; YORK, J. Amazon.com recommendations: item-to-item collaborative filtering. *IEEE Internet Computing*, v. 7, n. 1, p. 76-80, 2003.

SARWAR, B.; KARYPIS, G.; KONSTAN, J.; RIEDL, J. Item-based collaborative filtering recommendation algorithms. In: *Proceedings of the 10th International World Wide Web Conference*, 2001.

PAGE, L.; BRIN, S.; MOTWANI, R.; WINOGRAD, T. *The PageRank citation ranking: bringing order to the web*. Stanford InfoLab, 1999.

KLABNIK, S.; NICHOLS, C. *The Rust Programming Language*. No Starch Press. Disponível em: https://doc.rust-lang.org/book/.

RUST PROJECT. *std::collections*: documentação da biblioteca padrão. Disponível em: https://doc.rust-lang.org/std/collections/.